SmartCompany

Load Testing

Napredne veb tehnologije

Autori:
Ana Španović SV48/2021 (Student 1)
Andrija Slović SV12/2021 (Student 2)
Vojkan Marković SV65/2021 (Student 3)

1. Uvod i cilj testiranja

Cilj ovog izveštaja je da dokumentuje testiranje performansi (load testing) SmartCompany sistema pod rastućim brojem istovremenih korisnika, identifikuje uska grla u arhitekturi, opiše sprovedene optimizacije i izmeri njihov efekat, kao i da utvrdi realan kapacitet sistema na trenutnoj hardverskoj/infrastrukturnoj konfiguraciji.

Testiranje je sprovedeno alatom Locust (Python), simulirajući nekoliko tipova korisnika koji paralelno koriste različite funkcionalnosti sistema: kupce (Customer), menadžere magacina (Warehouse Manager), menadžere fabrika (Factory Manager) i menadžere zaduženih za vozila i firme (Vehicle/Company Manager). Testiranje pokriva ukupno 30 endpoint-a, po 10 po svakom studentu, u skladu sa specifikacijom projekta.

1.1 Specifikacija računara na kome je testiranje izvršeno

- CPU: Ryzen 5 5625U
- RAM: 32GB DDR4 3200Mhz
- SSD: NVMe
- OS: Windows 10

1.2 Arhitektura sistema

Sistem se sastoji od: Spring Boot backend aplikacije (Java 17, port 8080, context-path /api), Angular frontend aplikacije servirane preko nginx-a (port 80), PostgreSQL baze podataka (Docker), InfluxDB baze za vremenske serije (temperatura, proizvodnja, distanca vozila), Redis keša i RabbitMQ message brokera. Backend radi lokalno, dok su pomoćni servisi (Postgres, InfluxDB, Redis, RabbitMQ, nginx) pokrenuti kroz Docker Compose.

1.3 Metodologija testiranja

- Alat: Locust, sa realnim korisničkim tempom (wait_time 2–5s između akcija po korisniku), simulirajući stvarno ponašanje.
- Test scenariji pokrivaju kombinaciju GET (pretraga, detalji, statistika) i POST/PUT (kreiranje, izmena, odluka) operacija, uključujući operacije koje koriste pesimističko zaključavanje nad bazom.
- Testirani nivoi opterećenja: 10, 100, 250 i 500 istovremenih korisnika, sa umerenim spawn rate-om, u trajanju od 5 minuta po rundi (steady-state merenje).
- Oba korisnička tipa (Customer, Manager) testirani su simultano u istim rundama, kako bi se izmerilo realno, kombinovano opterećenje sistema.
- Za svaku rundu sačuvani su: CSV sa statistikom po endpoint-u, CSV sa listom grešaka (ako ih ima), i grafici iz Locust Web UI-a (Response Time, RPS, broj korisnika u vremenu).

2. Testirani endpoint-i

2.1 Student 2 - Narudžbine, magacini, korisnici (Customer + Warehouse Manager)

#	Metod	Putanja	Opis
1	POST	/api/login	Prijava korisnika (koristi se kao deo svakog scenarija)
2	GET	/api/users/check-username	Provera dostupnosti korisničkog imena
3	GET	/api/products	Pretraga proizvoda (stranica)
4	GET	/api/files/presigned-url/get	Dobijanje presigned URL-a za sliku proizvoda
5	GET	/api/orders/my-companies	Firme prijavljenog kupca
6	POST	/api/orders	Kreiranje porudžbine (pesimističko zaključavanje zaliha)
7	GET	/api/warehouses	Pretraga magacina
8	GET	/api/warehouses/{id}	Detalji magacina
9	PUT	/api/warehouses/{id}	Izmena magacina (pesimističko zaključavanje)
10	GET	/api/warehouses/{id}/sectors/{id}/temperature	Istorija temperature sektora (InfluxDB)

2.2 Student 1 - Fabrike, proizvodi, menadžeri (Factory Manager)

#	Metod	Putanja	Opis
1	GET	/api/factories	Pretraga fabrika
2	GET	/api/factories/{id}	Detalji fabrike
3	PUT	/api/factories/{id}	Izmena fabrike
4	GET	/api/factories/countries	Spisak država (fabrike)
5	GET	/api/factories/cities	Gradovi po državi
6	GET	/api/factories/{id}/products/{id}/production	Podaci o proizvodnji (InfluxDB)
7	GET	/api/users/managers	Pretraga menadžera (SuperAdmin)
8	PUT	/api/products/{id}	Izmena proizvoda
9	GET	/api/categories	Spisak kategorija
10	GET	/api/products/{id}	Detalji proizvoda

2.3 Student 3 - Vozila i zahtevi za registraciju firmi (Vehicle/Company Manager)

#	Metod	Putanja	Opis
1	GET	/api/vehicles	Pretraga vozila
2	GET	/api/vehicles/{id}	Detalji vozila
3	PUT	/api/vehicles/{id}	Izmena vozila
4	GET	/api/vehicles/brands	Spisak marki vozila
5	GET	/api/vehicles/models	Modeli po marki
6	GET	/api/vehicles/{id}/distance	Pređena distanca (InfluxDB)
7	GET	/api/companies	Pretraga zahteva za registraciju firmi
8	GET	/api/companies/{id}/manage	Detalji zahteva za menadžera
9	PUT	/api/companies/{id}/decision	Odluka o zahtevu (pesimističko zaključavanje + email)
10	GET	/api/companies/countries	Spisak država (firme)

3. Optimizacije primenjene tokom testiranja

Tokom iterativnog testiranja identifikovano je i rešeno više uskih grla. Optimizacije su primenjivane redom, i svaka je izmerena posebnom rundom testa pre prelaska na sledeću.

3.1 Infrastrukturne optimizacije (nginx, Tomcat, Docker)

- Podešen Tomcat thread pool (server.tomcat.threads.max=400, accept-count=200) za bolje podnošenje burst opterećenja pri prijavljivanju velikog broja korisnika odjednom.
- Podešen HikariCP connection pool sa podrazumevanih 10 na 90 konekcija (spring.datasource.hikari.maximum-pool-size=90), usklađeno sa PostgreSQL podešavanjem max_connections=100.
- Povećan shm_size Docker kontejnera za PostgreSQL (sa podrazumevanih 64MB na 1GB) - rešen problem "could not resize shared memory segment" koji se javljao pri teškim upitima (COUNT/sort nad tabelom firmi sa 2M+ redova) pod opterećenjem.

3.2 Asinhrono procesiranje sporih operacija

Slanje e-pošte i generisanje PDF faktura su sinhrone, spore operacije (SMTP round-trip) koje su, kada se izvršavaju unutar transakcije koja drži pesimistički zaključan red u bazi, drastično produžavale vreme držanja konekcije i zaključavanja.

- OrderService.createOrder() - razdvojen na kratku transakciju (ažuriranje zaliha) i asinhrono slanje PDF fakture (@Async), van transakcije. Efekat: medijana odgovora POST /api/orders pala je sa ~2900ms na ~250ms u ranim testovima.
- CompanyService.decide() - istim obrascem razdvojen na kratku transakciju (promena statusa zahteva) i asinhrono slanje mejla o odluci. Efekat: medijana odgovora PUT /api/companies/{id}/decision pala je sa ~2400ms ~300ms.
- Za potrebe load testiranja, slanje e-pošte je privremeno isključeno (app.email.enabled=false) da bi se izbeglo iscrpljivanje SMTP servisa i veštačko usporavanje testa; asinhrona arhitektura ostaje ista i sa uključenim slanjem.

3.3 Optimizacija pesimističkog zaključavanja zaliha (WarehouseService)

Prilikom kreiranja porudžbine, sistem je prvobitno uvek zaključavao SVE magacinske stavke (WarehouseItem) za dati proizvod, čak i kada je jedan magacin sam imao dovoljno zaliha za celu porudžbinu. Pod konkurentnim opterećenjem (mnogo kupaca kupuje isti proizvod istovremeno), ovo je stvaralo veliku kontenciju.

- Dodata je logika koja prvo pokušava da pronađe jedan magacin sa dovoljnom količinom i zaključa samo njega; tek ako nijedan pojedinačni magacin nema dovoljno, sistem "pada nazad" na skupljanje iz više magacina (stari, sveobuhvatniji pristup).
- Dodatno, u load test scenariju uveden je nasumičan izbor proizvoda (umesto uvek istog) i realističniji tempo korisnika (wait_time 2–5s, manji udeo create_order akcije), čime je smanjena veštačka kontencija specifična za test, ne za realnu upotrebu.
- Efekat kombinovane optimizacije: medijana POST /api/orders na 500 korisnika pala je sa 26.000–42.000ms (pre optimizacije) na ~790ms (posle), uz 0 grešaka.

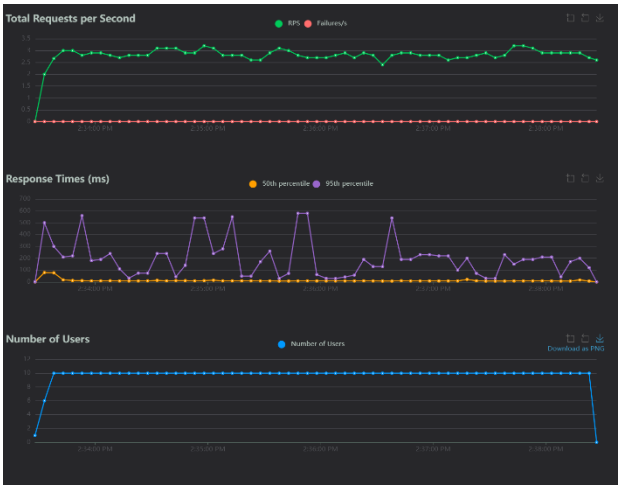
3.4 Zaključak o optimizacijama

Sve gorenavedene optimizacije su kumulativno primenjene pre finalne serije merenja (10/100/250/500 korisnika) prikazane u sledećim poglavljima. Rezultati u poglavljima 4–6 stoga odražavaju stanje sistema nakon optimizacija.

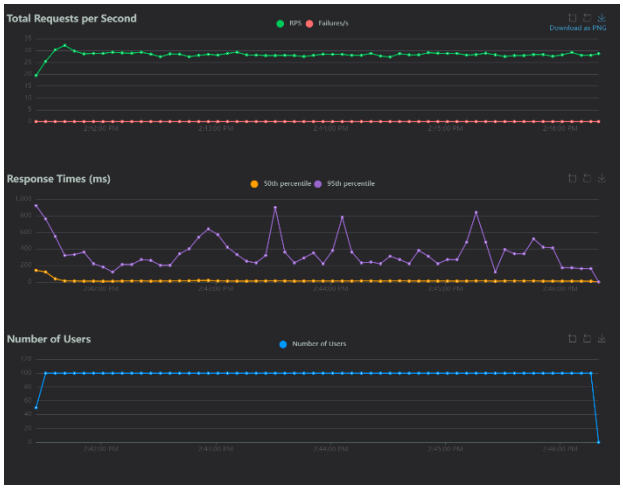
4. Ukupni (agregatni) rezultati testiranja

Sledeća tabela prikazuje agregatne rezultate za sve testirane endpoint-e (sva tri studenta zajedno, svih 30 endpoint-a) po nivou opterećenja. Svi nivoi su testirani sa svim tipovima korisnika aktivnih istovremeno, što predstavlja realno, kombinovano opterećenje sistema.

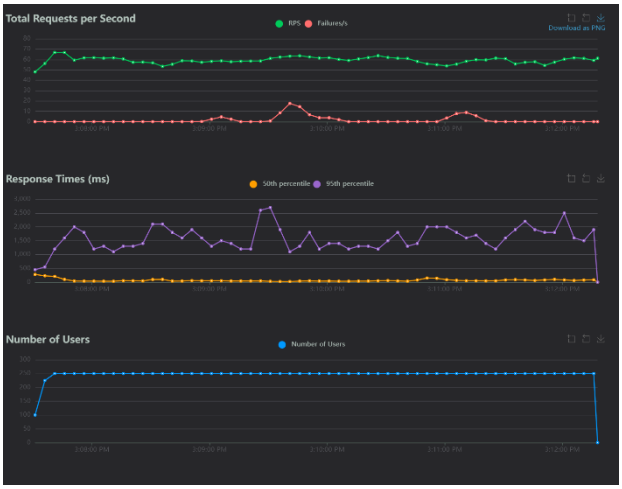
Korisnika	Ukupno zahteva	Greške	% grešaka	Medijana (ms)	Prosek (ms)	Max (ms)
10	858	0	0%	8	40,6	620
100	8 482	0	0%	11	73,8	840
250	17 245	471	2,73%	57	699,7	60 056
500	22 177	6 959	31,39%	82	1 806,2	61 023



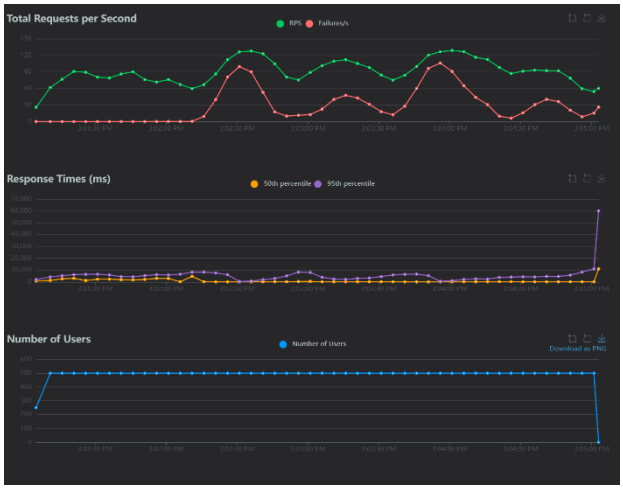
Grafici za test sa 10 korisnika



Grafici za test sa 100 korisnika



Grafici za test sa 250 korisnika



Grafici za test sa 500 korisnika

4.1 Tumačenje rezultata

- 10 i 100 korisnika: sistem radi besprekorno - 0% grešaka, medijana odgovora ispod 15ms na oba nivoa. Ovo potvrđuje da su optimizacije iz poglavlja 3 uspešno rešile probleme identifikovane u ranijim (neoptimizovanim) rundama testiranja.
- 250 korisnika: pojavljuje se prvi, umeren nivo grešaka (2,73%), gotovo isključivo u vidu 502 Bad Gateway grešaka i produženih vremena odgovora na endpoint-ima koji rade teže upite (npr. GET /api/companies sa COUNT nad tabelom od 2M+ redova, PUT /api/companies/{id}/decision).
- 500 korisnika: sistem dostiže granicu kapaciteta - 31,4% zahteva završava greškom, dominantno 502 Bad Gateway (299 od 328 grešaka) i 504 (gateway timeout).
- Obrazac degradacije (nagli skok grešaka između 100 i 500 korisnika, sa 250 kao prelaznom tačkom) je konzistentan sa ranije identifikovanim uzrocima: zasićenje HikariCP connection pool-a (90 konekcija, ograničeno PostgreSQL podešavanjem max_connections=100) pod kombinovanim opterećenjem svih pet korisničkih tipova, koji dele isti pool.

5. Detaljna analiza - Narudžbine i magacini (Student 2)

Ovaj deo pokriva funkcionalnosti kupovine (kreiranje porudžbina, pregled proizvoda i firmi kupca) i upravljanja magacinima (pregled, izmena, praćenje temperature).

Analiza po endpoint-u

POST /api/orders - nakon primenjenih optimizacija (single-warehouse-first zaključavanje, async faktura), performanse su odlične do 100 korisnika (medijana ~330ms na 100 korisnika, sa velikim brojem korisnika koji kupuju isti mali skup proizvoda). Degradacija na 250+ korisnika prati opšti obrazac zasićenja connection pool-a, ne specifičan problem porudžbina.

GET /api/warehouses i GET /api/warehouses/{id} - najčešće pozivani endpoint-i u ovoj grupi, pa su i najviše izloženi zasićenju pool-a na 250/500 korisnika. Ovo je očekivano - endpoint sa najvećim brojem poziva prvi "oseti" zasićenje deljenog resursa.

GET /api/warehouses/{id}/sectors/{id}/temperature (InfluxDB upit) - pokazuje dobru skalabilnost nezavisno od PostgreSQL connection pool problema, jer koristi drugu bazu podataka; medijana ostaje u granicama 16–63ms na svim nivoima opterećenja.

POST /api/login					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	10	0%	107	76	355
100	100	0%	198	120	512
250	250	0%	430	163	919
500	500	2.1%	2700	8	8639

GET /api/users/check-username					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	44	0%	8	6	12
100	332	0%	9	5	58
250	667	1.9%	47	4	519

GET /api/users/check-username					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
500	942	29.2%	1722	3	10 118

GET /api/products					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	62	0%	35	6	126
100	549	0%	116	5	32293
250	1234	3.0%	585	3	60025
500	1588	28.7%	1479	3	10 088

GET /api/orders/my-companies					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	49	0%	32	24	55
100	421	0%	49	23	329
250	1003	2.5%	392	3	60 010
500	1267	29.2%	2232	3	10 459

POST /api/orders					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	17	0%	211	171	305
100	106	0%	355	194	1312
250	258	3.1%	1323	4	60 001
500	300	27.3%	3050	4	13 608

GET /api/files/presigned-url/get					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	78	0%	8	5	126
100	681	0%	9	4	238
250	1536	1.6%	34	4	342
500	1946	29.6%	95	3	60 010

GET /api/warehouses					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	80	0%	18	6	501
100	869	0%	22	5	864
250	1878	2.8%	215	3	15 690
500	2590	33.4%	1126	3	11 182

GET /api/warehouses/100					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	49	0%	9	7	17
100	649	0%	13	6	357
250	1480	3.5%	137	3	59 996
500	1815	33.6%	1867	3	60 004

PUT /api/warehouses/100					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	12	0%	19	15	38
100	181	0%	31	11	869
250	535	3.2%	145	3	2698
500	579	33.0%	1948	3	12262

GET /api/warehouses/100/sectors/99983/temperature					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	28	0%	13	10	20
100	403	0%	19	10	389
250	939	2.9%	63	3	728
500	1208	31.2%	97	3	3301

6. Detaljna analiza - Fabrike i proizvodi (Student 1)

Ovaj deo pokriva pregled i upravljanje fabrikama, praćenje proizvodnje (InfluxDB), i deo funkcionalnosti proizvoda i kategorija koje koristi manager uloga.

Analiza po endpoint-u

GET /api/factories/{id}/products/{id}/production (InfluxDB) - kao i kod magacina, InfluxDB upiti pokazuju bolju otpornost na PostgreSQL zasićenje; medijana ostaje ispod 50ms čak i na 250 korisnika, degradacija tek na 500.

PUT /api/factories/{id} - endpoint sa pisanjem u bazu (najosetljiviji na zaključavanje/connection pool), pokazuje najveći procenat grešaka na 500 korisnika u ovoj grupi (43,4%), potvrđujući da write operacije prve osećaju zasićenje pool-a.

GET /api/products/{id} - vraća relativno veliki payload, što dodatno opterećuje mrežni sloj pod velikim brojem konkurentnih zahteva; vredi razmotriti odvajanje slike od osnovnih podataka proizvoda u budućim iteracijama.

GET /api/factories					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	50	0%	27	5	137
100	348	0%	63	5	970

GET /api/factories					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
250	895	3.7%	466	3	2289
500	1112	33.0%	1433	3	59 998

GET /api/factories/1					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	26	0%	21	15	94
100	312	0%	27	15	125
250	649	3.4%	186	3	15 950
500	842	29.1%	2000	3	10 166

PUT /api/factories/1					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	10	0%	36	31	59
100	121	0%	56	31	473
250	198	1.0%	383	8	3000
500	283	30.4%	2446	3	12 206

GET /api/factories/countries					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	21	0%	7	6	11
100	200	0%	9	4	39
250	409	2.7%	45	4	342
500	604	29.0%	87	3	2329

GET /api/factories/cities					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	31	0%	7	5	9
100	201	0%	11	5	642
250	483	4.8%	44	3	535
500	577	28.8%	189	3	60 032

GET /api/factories/1/products/2/production					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	39	0%	9	6	35
100	251	0%	10	5	72
250	614	2.9%	50	3	653
500	860	30.6%	1934	3	60 004

GET /api/users/managers					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	25	0%	11	6	73
100	208	0%	12	5	252
250	442	4.1%	41	3	315
500	534	29.2%	112	3	4530

PUT /api/products/2					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	11	0%	28	18	83
100	77	0%	35	16	543
250	228	1.3%	227	4	2243
500	262	28.2%	2460	3	11 996

GET /api/categories					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	17	0%	7	5	9
100	198	0%	9	4	150
250	407	3.4%	414	3	60 005
500	536	28.9%	92	3	2039

GET /api/products/2					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	25	0%	165	40	335
100	181	0%	280	35	1394
250	428	4.2%	945	3	2798
500	533	28.0%	97	3	10 454

7. Detaljna analiza - Vozila i zahtevi za registraciju firmi (Student 3)

Ovaj deo pokriva upravljanje dostavnim vozilima (pregled, izmena, praćenje pređene distance) i obradu zahteva za registraciju firmi (pregled, odluka menadžera). PUT /api/companies/{id}/decision je posebno zanimljiv jer kombinuje pesimističko zaključavanje sa asinhronim slanjem e-pošte - optimizacija ovog endpoint-a je detaljno opisana u poglavlju 3.2.

Analiza po endpoint-u

PUT /api/companies/{id}/decision - nakon async optimizacije, medijana odgovora na 100 korisnika je odlična (~127ms), potvrđujući da je asinhrono slanje e-pošte uspešno rešilo prethodno uočeni problem sinhronog slanja unutar transakcije.

GET /api/companies?status=PENDING - endpoint koji svaki poziv PUT decision prvo koristi da pronađe dostupan zahtev; ima nešto veće vreme odgovora (140–4500ms u zavisnosti od opterećenja) jer filtrira po statusu nad tabelom sa 2M+ redova

GET /api/vehicles/{id}/distance (InfluxDB) - slično kao kod temperature/proizvodnje, pokazuje stabilne performanse (13–61ms medijana) sve do 500 korisnika, gde tek počinje degradacija usled opšteg zasićenja sistema, ne usled samog InfluxDB upita.

GET /api/vehicles					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	21	0%	17	6	65
100	361	0%	17	5	503
250	430	3.0%	114	4	1044
500	599	36.2%	1042	3	10 775

GET /api/vehicles/1					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	26	0%	17	6	65
100	240	0%	17	5	503
250	353	2.0%	114	4	1044
500	460	36.2%	1042	3	10 775

PUT /api/vehicles/1					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	7	0%	13	11	20
100	76	0%	14	8	49
250	105	1.0%	99	3	625
500	120	28.3%	1832	3	9161

GET /api/vehicles/brands					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	17	0%	7	5	11
100	199	0%	9	4	107
250	210	0.5%	46	7	321
500	295	39.7%	98	3	2286

GET /api/vehicles/models					
Broj korisnika	**Broj zahteva**	**Broj grešaka**	**Avg response (ms)**	**Min response (ms)**	**Max response (ms)**
10	16	0%	7	6	8
100	169	0%	9	5	215
250	320	0.4%	183	3	32 492

GET /api/vehicles/models					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
500	287	36.2%	346	3	60 047

GET /api/vehicles/1/distance					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	21	0%	17	11	93
100	241	0%	17	9	87
250	340	2.9%	61	5	457
500	404	39.4%	241	3	60 004

GET /api/companies					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	29	0%	417	130	620
100	346	0%	805	147	1832
250	445	1.8%	16 642	4	40 615
500	473	64.1%	24554	3	61023

GET /api/companies/1/manage					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	13	0%	69	56	95
100	184	0%	126	61	442
250	237	3.0%	2557	3	5189
500	277	33.2%	6084	3	16 001

PUT /api/companies/{id}/decision					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	10	0%	74	62	88
100	94	0%	116	10	771
250	104	1.0%	1653	18	60 056
500	75	21.3%	3540	4	12360

GET /api/products/2					
Broj korisnika	Broj zahteva	Broj grešaka	Avg response (ms)	Min response (ms)	Max response (ms)
10	14	0%	8	5	21
100	184	0%	9	5	104
250	249	1.6%	38	4	268
500	291	35.7%	117	3	2584

8. Zaključak i preporuke

8.1 Sumarni zaključak

Sistem SmartCompany, nakon primenjenih optimizacija (asinhrono procesiranje sporih operacija, optimizovano pesimističko zaključavanje, podešeni Tomcat i HikariCP parametri, ispravke Docker/nginx konfiguracije), pouzdano podnosi kombinovano opterećenje do 100 istovremenih korisnika svih pet tipova (Customer, Warehouse/Factory/Vehicle Manager, Company Manager) bez ijedne greške. Na 250 korisnika počinje umerena degradacija (2,7% grešaka), a na 500 korisnika sistem dostiže granicu kapaciteta trenutne konfiguracije (31,4% grešaka).

Dominantan uzrok degradacije na 250+ korisnika je zasićenje deljenog PostgreSQL connection pool-a (HikariCP, 90 konekcija, ograničeno server podešavanjem max_connections=100) pod kombinovanim opterećenjem svih modula tima. Endpoint-i koji koriste InfluxDB (temperatura, proizvodnja, distanca vozila) pokazuju znatno bolju otpornost na opterećenje, jer ne dele isti resurs sa PostgreSQL operacijama.

8.2 Predlozi za dalja poboljšanja (nerealizovano)

- Uvođenje connection poolera (PgBouncer) ispred PostgreSQL baze radi efikasnijeg deljenja konekcija između većeg broja konkurentnih korisnika.
- Povećanje PostgreSQL max_connections uz odgovarajuće resurse (RAM/CPU) na DB serveru, ili horizontalno skaliranje sa read-replikama za GET upite.
- Razmatranje keširanja za GET /api/companies (trenutno namerno nekeširano zbog zahteva za ažurnošću podataka za menadžere) - možda kraći TTL keš (npr. 5-10 sekundi) kao kompromis.
- Testiranje na odvojenim mašinama za klijenta (Locust) i server, umesto na jednoj razvojnoj mašini — eliminisalo bi deljenje CPU/mrežnih resursa između testa i sistema koji se testira, dajući precizniju sliku stvarnog kapaciteta servera.

8.3 Napomena o metodologiji

Sve runde testiranja izvršene su na jednoj razvojnoj mašini (i klijent/Locust i server dele isti hardver), što znači da deo izmerene degradacije na višim nivoima opterećenja može delimično poticati i od ograničenja same test mašine, a ne isključivo od aplikacije.